UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026).
   Answers are unofficial reference answers, not official UTM marking schemes.

2. Marking points show the expected answer and mark allocation; check them against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Approved real questions only; no generators, placeholders or parametric variants.
   Questions and answers are for local study use only.

5. Structure derived from the supplied CSC429 papers; every question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 11 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION 1 (5 marks)

Source: CSC429-2023-07, page [2]

Unofficial model answer:

a) The Von Neumann architecture consists of a CPU (containing the control unit and ALU), memory, input devices, output devices, and secondary storage, all connected by a shared bus system. The control unit fetches and decodes instructions, the ALU performs arithmetic and logic operations, memory holds both instructions and data, input devices supply data (e.g., keyboard, mouse), output devices present results (e.g., monitor, printer), and secondary storage provides non-volatile storage (e.g., hard disk, SSD). The key feature is the stored-program concept where instructions and data reside in the same memory and are accessed via a shared bus, leading to the Von Neumann bottleneck. A diagram should show these components with arrows indicating data/address/control flow.

b) The actual device that John Von Neumann himself developed is the IAS (Institute for Advanced Study) machine, also known as the Von Neumann machine. It was built at the Institute for Advanced Study in Princeton and is considered the first stored-program computer.

Marking points:

• 1 mark: Correctly drawn block diagram showing CPU (control unit and ALU), memory, input, output, and secondary storage connected by buses.

• 1 mark: Relevant example given for each component (e.g., keyboard for input, monitor for output, hard disk for secondary storage, etc.).

• 1 mark: Clear indication of the stored-program concept or shared bus for instructions and data.

• 1 mark: Correctly naming the IAS machine (or Von Neumann machine) as the device developed by John Von Neumann.

• 1 mark: Additional detail such as the location (Institute for Advanced Study) or its significance as the first stored-program computer.

Model confidence: high.

Model note: The question asks to 'draw' the architecture; the answer describes the diagram. The marking scheme assumes a typical block diagram is expected. The device in part (b) is commonly known as the IAS machine; some sources may refer to it as the 'Von Neumann machine' or 'IAS computer'.

QUESTION 2 (5 marks)

Source: CSC429-2016-06, page [3]

Unofficial model answer:

a) AI was introduced in the fifth generation of computers. The fifth generation (from the 1980s onward) is characterised by parallel processing, very-large-scale integration (VLSI), and a focus on artificial intelligence and natural-language processing.

b) The programming language most associated with this technology is Prolog (PROgramming in LOGic), a logic programming language widely used for AI and expert systems. LISP is also acceptable as an early AI language, but Prolog is the standard answer for fifth-generation AI.

c) The advantage of embedding AI in everyday life is that it automates routine and complex tasks, improving convenience, efficiency and decision-making. For example, smart-home systems adjust lighting and temperature automatically, intelligent tutoring systems personalise learning, and robots perform dangerous or repetitive jobs, thereby saving time and reducing human error.

d) One possible scenario where AI can harm people is an autonomous vehicle or robot malfunctioning due to a

© Hak Cipta Universiti Teknologi MARA

software error or flawed training data, causing a collision or physical injury. Another scenario is an AI-based medical diagnosis system giving an incorrect recommendation that leads to wrong treatment. The key point is that AI lacks human judgement and accountability, so errors can directly endanger safety or privacy.

Marking points:

• *1 mark: (a) Fifth generation stated.*

• *1 mark: (b) Prolog (or LISP) named as the AI programming language.*

• *1 mark: (c) Valid advantage of embedding AI (e.g., automation, convenience, efficiency, personalisation) with a brief explanation.*

• *2 marks: (d) One plausible harmful scenario clearly described (1 mark for the scenario, 1 mark for explaining how it harms people).*

Model confidence: high.

Model note: The question text contains a typo 'A1' for 'AI'. The answer assumes standard CSC429 course material where the fifth generation is associated with AI and Prolog is the representative language. No figure or table was supplied.

QUESTION 3 (5 marks)

Source: CSC429-2023-02, page [2]

Unofficial model answer:

The question text is garbled by OCR, so the intended operands are reconstructed from context: (a) is a binary subtraction $1.00001001_2 - 0.00110001_2$ (the 'OOH' is read as 0011 0001), and (b) is a hexadecimal subtraction $0.FA3_{16} - 0.651_{16}$ (the 'i6' is read as subscript 16 and '651s' as 651_{16}). Both are fractional subtractions, so the workflow is: align radix points, pad with zeros, subtract column by column with borrows, then state the result.

(a) Binary subtraction $1.00001001_2 - 0.00110001_2$. Align both numbers to 8 fractional bits: minuend 1.00001001, subtrahend 0.00110001. Working from the least significant bit (rightmost): bit 8: 1−1=0; bit 7: 0−0=0; bit 6: 0−0=0; bit 5: 0−1 requires a borrow — borrow from bit 4, giving 10_2−1=1, and bit 4 becomes 0; bit 4: now 0−1 again requires a borrow from bit 3, giving 10_2−1=1, bit 3 becomes 0; bit 3: 0−1 borrow from bit 2, giving 1, bit 2 becomes 0; bit 2: 0−1 borrow from bit 1, giving 1, bit 1 becomes 0; bit 1: 0−0=0; integer bit: 1−0=1. Result: 1.00010000_2, i.e. 1.0001_2 (decimal check: 1.03515625 − 0.19140625 = 0.84375 = 1.0001_2 is not equal; the correct decimal check is 1.00001001_2 = 1.03515625 and 0.00110001_2 = 0.19140625, difference 0.84375 = 0.11011_2, so the borrow chain above must be rechecked). Corrected column-by-column working: 1.00001001 − 0.00110001. Bit 8: 1−1=0. Bit 7: 0−0=0. Bit 6: 0−0=0. Bit 5: 0−1: borrow from bit 4 (which is 0, so borrow propagates to bit 3, bit 2, bit 1, then the integer 1). The integer 1 becomes 0, bit 1 becomes 10_2, lends 1 to bit 2 leaving 1, bit 2 lends to bit 3 leaving 1, bit 3 lends to bit 4 leaving 1, bit 4 lends to bit 5 leaving 1, and bit 5 becomes 10_2. Then bit 5: 10−1=1; bit 4: 1−1=0; bit 3: 1−0=1; bit 2: 1−0=1; bit 1: 1−0=1; integer: 0−0=0. Result: $0.11011000_2 = 0.11011_2 = 0.84375_{10}$, which matches the decimal check. So the answer is 0.11011_2 (0.11011000_2).

(b) Hexadecimal subtraction $0.FA3_{16} - 0.651_{16}$. Align to three hex digits: FA3 − 651. Least significant digit: 3−1=2. Middle digit: A−5 = 10−5 = 5. Most significant digit: F−6 = 15−6 = 9. Result: 0.952_{16}. Decimal check: $0.FA3_{16}$ = 15/16 + 10/256 + 3/4096 = 0.9375 + 0.0390625 + 0.000732421875 = 0.977294921875; 0.651_{16} = 6/16 + 5/256 + 1/4096 = 0.375 + 0.01953125 + 0.000244140625 = 0.394775390625; difference = 0.58251953125 = 9/16 + 5/256 + 2/4096 = 0.5625 + 0.01953125 + 0.00048828125 = 0.58251953125, confirming 0.952_{16}.

Marking points:

• *1 mark: correct alignment of radix points and zero-padding for (a)*

• *1 mark: correct borrow-propagating subtraction workflow for (a) with result 0.11011_2 (accept 0.11011000_2)*

• *1 mark: correct digit-by-digit hexadecimal subtraction for (b) with result 0.952_{16}*

• *1 mark: correct handling of the hex letters (A=10, F=15) and no spurious borrow in (b)*

© Hak Cipta Universiti Teknologi MARA

• 1 mark: verification by decimal conversion (or equivalent check) for at least one part, with clear workflow shown

Model confidence: medium.

Model note: The printed question is corrupted by OCR: '1.0000 10012-0.0011 OOH2' and 'O.FA3i6-0.651s' are ambiguous. I assumed (a) is $1.00001001_2 - 0.00110001_2$ and (b) is $0.FA3_{16} - 0.651_{16}$, based on the '2' and '16' subscripts and the hex letters F, A. If the intended operands differ (e.g. 0.00110011_2 or 0.651_8), the numerical results change; the method and marking allocation remain valid. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 4 (5 marks)

Source: CSC429-2014-06, page [5]

Unofficial model answer:

a) The addressing technique illustrated in Figure 2 is **direct addressing** (also called absolute addressing). The instruction contains an Opcode plus an Address field A, and that address A is used directly as the memory address of the operand — the operand is fetched from memory location A with no further calculation.

b) How it is used: The CPU decodes the instruction, takes the address field A from the instruction, and places A directly on the memory address bus (MAR ← A). The memory is then read and the operand stored at location A is transferred to the data bus and into the CPU (MDR/accumulator). The effective address is simply EA = A, so no address arithmetic or index/pointer dereferencing is required; the operand is accessed in one memory reference after instruction fetch.

c) Advantage: It is simple and fast — the effective address is obtained directly from the instruction with no extra address computation, and only one operand memory access is needed (good for accessing fixed/global variables). Disadvantage: The address field must be large enough to hold a full memory address, so the instruction is long (limited addressable range for a given instruction word length), and it is inflexible — the address is fixed at assembly time, so it cannot easily support relocation, arrays, or pointer/indexed access.

Marking points:

• 1 mark (a): Correctly names direct addressing / absolute addressing.

• 2 marks (b): Explains that address field A is used directly as the operand's memory address (EA = A); 1 mark for stating the operand is fetched from memory location A, 1 mark for noting no address calculation/one memory access after fetch.

• 1 mark (c): One valid advantage (e.g., simple/fast, no address computation, single operand fetch).

• 1 mark (c): One valid disadvantage (e.g., long instruction/large address field, limited address range, inflexible/fixed address, poor for arrays/relocation).

Model confidence: high.

Model note: Figure 2 is supplied only by caption/extracted text; the answer assumes the standard diagram of an instruction with Opcode + Address A pointing to a memory operand, which is the conventional depiction of direct addressing. If the actual figure showed an indirect pointer or index register, the label would differ; this cannot be verified from the caption alone. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 5 (10 marks)

Source: CSC429-2015-06, page [4]

Unofficial model answer:

a) The keyboard provides two types of characters: (1) Printable (alphanumeric) characters – these are the visible characters such as letters (A–Z, a–z), digits (0–9), punctuation marks, and symbols (e.g., @, #, $). They are encoded using a character code (e.g., ASCII) and are stored/displayed as data. (2) Control characters – these are non-printable characters that perform control functions, such as Enter (carriage return/line feed), Backspace, Delete, Tab, Esc, and function keys. They are also represented by unique codes (e.g., ASCII control codes) but are interpreted by the system to control the cursor, editing, or program flow rather than being displayed as glyphs.

© Hak Cipta Universiti Teknologi MARA

b) Three categories of input/output modules are: (1) Human-readable – devices that allow humans to communicate with the computer, e.g., keyboard (input) and monitor/display (output). (2) Machine-readable – devices that communicate directly with equipment, e.g., disk drive (magnetic disk) or sensor/actuator. (3) Communication – devices that enable the computer to communicate with remote devices over a network, e.g., network interface card (NIC) or modem.

Marking points:

• 1 mark: Correctly identifies printable/alphanumeric characters with a valid example (e.g., letters, digits, symbols).

• 1 mark: Correctly identifies control characters with a valid example (e.g., Enter, Backspace, Tab, Esc).

• 1 mark: Briefly explains the difference in function (printable are displayed/stored as data; control perform actions).

• 1 mark: Correctly names category 1: Human-readable.

• 1 mark: Gives an appropriate example for human-readable (e.g., keyboard, monitor).

• 1 mark: Correctly names category 2: Machine-readable.

• 1 mark: Gives an appropriate example for machine-readable (e.g., disk drive, sensor).

• 1 mark: Correctly names category 3: Communication.

• 1 mark: Gives an appropriate example for communication (e.g., NIC, modem).

• 1 mark: Overall clarity and correct terminology (e.g., ASCII, control codes, I/O module).

Model confidence: high.

Model note: The question does not specify a particular character encoding; ASCII is assumed as the common example. The categories of I/O modules are based on standard textbook classifications (e.g., Stallings). No figure or table was provided, so the answer is based solely on the question text.

QUESTION 6 (10 marks)

Source: CSC429-2025-07, page [4]

Unofficial model answer:

a) SISD architecture diagram: The SISD (Single Instruction, Single Data) architecture consists of a Control Unit (CU), a Processing Unit (PU), and a Memory Unit (MU). The CU fetches instructions from memory via the instruction stream (IS) and issues control signals to the PU. The PU executes the instruction and accesses data from memory via the data stream (DS). A typical block diagram: MU is connected to CU and PU; CU sends control signals to PU; IS flows from MU to CU; DS flows between MU and PU. (Diagram should show: MU ↔ CU (IS), MU ↔ PU (DS), CU → PU (control)).

b) Two examples of shared-memory MIMD: (1) Multicore processors (e.g., Intel Core i7, AMD Ryzen) where multiple cores share a common main memory. (2) Symmetric Multiprocessor (SMP) systems, such as a server with multiple CPUs sharing the same RAM.

c) Difference between virus attack on shared-memory MIMD vs distributed-memory MIMD: In shared-memory MIMD, all processors access a single global memory, so a virus can spread rapidly by infecting shared data or code in the common memory, potentially compromising all processors and the entire system. In distributed-memory MIMD, each processor has its own private memory and communicates via message passing; a virus would be confined to the infected node's local memory and would need to explicitly send malicious messages to other nodes to propagate, making containment easier and limiting the spread.

d) One benefit of distributed-memory MIMD: Scalability. Example: A cluster of commodity computers (e.g., Beowulf cluster) can be easily expanded by adding more nodes, each with its own memory, to increase processing power without the bottleneck of shared memory contention. This allows cost-effective scaling for large-scale scientific simulations.

Marking points:

• 1 mark: Correctly draws SISD diagram with CU, PU, MU labeled.

• 1 mark: Shows instruction stream (IS) from MU to CU and data stream (DS) between MU and PU.

• 1 mark: Shows control signals from CU to PU.

© Hak Cipta Universiti Teknologi MARA

• 1 mark: Lists one correct example of shared-memory MIMD (e.g., multicore processor).

• 1 mark: Lists a second correct example (e.g., SMP server).

• 1 mark: Explains that in shared-memory MIMD, virus can spread via common memory to all processors.

• 1 mark: Explains that in distributed-memory MIMD, virus is confined to local node and requires message passing to spread.

• 1 mark: States one benefit of distributed-memory MIMD (e.g., scalability).

• 1 mark: Provides a relevant example (e.g., cluster computing).

• 1 mark: Clearly contrasts the two scenarios or provides a coherent discussion.

Model confidence: medium.

Model note: The SISD diagram is described textually because the original figure cannot be rendered; the answer assumes a standard block diagram. The marking scheme is unofficial and based on typical exam expectations. The virus discussion is open-ended; other valid points (e.g., ease of recovery, performance impact) could be accepted.

QUESTION 7 (10 marks)

Source: CSC429-2015-06, page [5]

Unofficial model answer:

a) Four classifications of multiple-processor architecture: (1) Symmetric Multiprocessor (SMP) — all processors share one main memory and have equal access/status, typically connected via a shared bus; (2) Non-Uniform Memory Access (NUMA) — multiple processors/nodes each with local memory, where access time to local memory is faster than to remote memory; (3) Cluster — a set of independent computers (nodes) connected by a network working together as a single system; (4) Massively Parallel Processor (MPP) / distributed-memory multicomputer — many processors each with private memory communicating by message passing. (Other acceptable classifications: vector/SIMD processors, multicore, UMA vs NUMA vs COMA memory models, Flynn's SISD/SIMD/MISD/MIMD.)

b) X = Shared-Memory / tightly-coupled multiple-processor architecture (i.e., multiple processors that share a common memory address space). SMP and NUMA are shared-memory (tightly coupled) systems; Cluster is loosely coupled, so the intended grouping is the shared-memory multiprocessor family. (If the lecturer's taxonomy groups all three, X = multiple-processor / parallel architecture; the shared-memory interpretation is the standard one.)

c) Differentiation: SMP has all processors sharing a single physical memory and I/O, connected by a common bus, with uniform memory access time and one operating system image; it is tightly coupled and scales poorly beyond a modest number of processors due to bus/memory contention. A Cluster consists of separate, autonomous computers (each with its own CPU, memory and OS) linked by a high-speed network; it is loosely coupled, has no shared global memory (communication is by message passing), and scales well by adding nodes. Opinion/justification: SMP is easier to manage and control because there is a single OS image, one memory space, uniform access, simpler programming model and centralized administration; a cluster requires distributed OS/middleware, message-passing software, and per-node management, making it harder to administer and program despite better scalability and fault isolation.

Marking points:

• 1 mark: SMP correctly listed/defined (shared single memory, equal processors)

• 1 mark: NUMA correctly listed/defined (local vs remote memory access times differ)

• 1 mark: Cluster correctly listed/defined (independent nodes over a network)

• 1 mark: fourth valid classification (e.g., MPP/distributed-memory, vector/SIMD, multicore, Flynn MIMD)

• 1 mark: X correctly identified as shared-memory / tightly-coupled multiprocessor architecture

• 1 mark: SMP described as tightly coupled, single shared memory, uniform access, common bus

• 1 mark: Cluster described as loosely coupled, independent nodes, message passing, no shared global memory

• 1 mark: clear contrast (coupling, memory model, scalability, communication)

• 1 mark: justified opinion on which is easier to manage (single OS image / uniform memory / simpler programming

© Hak Cipta Universiti Teknologi MARA

for SMP)

• 1 mark: coherent justification tied to management and control (administration, programming model, scalability trade-off)

Model confidence: medium.

Model note: Part (b) is ambiguous: the paper's 'X' likely refers to the shared-memory/tightly-coupled family (SMP+NUMA), but Cluster is loosely coupled, so if the lecturer's taxonomy intends all three, X = multiple-processor/parallel architecture. Answer states the standard interpretation and flags the alternative. No figure supplied.

QUESTION 8 (10 marks)

Source: CSC429-2019-12, page [3]

Unofficial model answer:

Subpart (i): Convert 1210.33_{10} to hexadecimal. The integer part 1210_{10} = 4×256 + 11×16 + 10 = 0x4BA (since 1210 ÷ 16 = 75 remainder 10 (A), 75 ÷ 16 = 4 remainder 11 (B), 4 ÷ 16 = 0 remainder 4). The fractional part 0.33_{10} is converted by repeated multiplication by 16: 0.33×16 = 5.28 → digit 5, 0.28×16 = 4.48 → digit 4, 0.48×16 = 7.68 → digit 7, 0.68×16 = 10.88 → digit A, 0.88×16 = 14.08 → digit E, 0.08×16 = 1.28 → digit 1, … giving $0.547AE1\ldots_{16}$. Thus $1210.33_{10} \approx 4BA.547AE1\ldots_{16}$ (or $4BA.547AE_{16}$ to 5 fractional hex digits). Subpart (ii): Convert 295.8125_{10} to octal. Integer part: 295 ÷ 8 = 36 remainder 7, 36 ÷ 8 = 4 remainder 4, 4 ÷ 8 = 0 remainder 4, so $295_{10} = 447_8$. Fractional part: 0.8125×8 = 6.5 → digit 6, 0.5×8 = 4.0 → digit 4, remainder 0, so $0.8125_{10} = 0.64_8$ exactly. Therefore $295.8125_{10} = 447.64_8$. Subpart (iii): Convert $BACA_{16}$ to binary. Each hexadecimal digit maps to four binary bits: B = 1011, A = 1010, C = 1100, A = 1010. Concatenating gives $BACA_{16}$ = 1011 1010 1100 1010_2 = 1011101011001010_2.

Marking points:

• 1 mark: correct integer conversion of 1210_{10} to $4BA_{16}$ with division-by-16 work shown

• 2 marks: correct fractional conversion of 0.33_{10} by repeated multiplication by 16, showing at least 4 hex digits (547A…) and stating the result is approximate/truncated

• 1 mark: correct final answer $1210.33_{10} \approx 4BA.547A\ldots_{16}$ (or equivalent with rounding)

• 1 mark: correct integer conversion of 295_{10} to 447_8 with division-by-8 work shown

• 1 mark: correct fractional conversion of 0.8125_{10} to 0.64_8 by repeated multiplication by 8, showing termination

• 1 mark: correct final answer $295.8125_{10} = 447.64_8$

• 2 marks: correct 4-bit binary mapping for each hex digit B→1011, A→1010, C→1100, A→1010

• 1 mark: correct concatenated binary result 1011101011001010_2 (spacing optional)

Model confidence: high.

Model note: The subscripts in the question text appear as '1210.331s', '295.8125io', and 'BACAieto'; these are interpreted as base-10 ($_{10}$), base-10 ($_{10}$), and base-16 ($_{16}$) respectively, consistent with the topic and typical notation. The fractional conversion of 0.33_{10} is non-terminating; the answer gives a truncated/rounded hexadecimal expansion. No figure or table was supplied.

QUESTION 9 (10 marks)

Source: CSC429-2016-12, page [3]

Unofficial model answer:

a) The size of a bus is not determined solely by the word size. The word size (64 bits) typically refers to the data bus width, so the data bus would be 64 bits wide. However, the overall bus size also includes the address bus width (which depends on the maximum memory addressable) and control lines. Therefore, the data bus size is 64 bits, but the total bus width is not specified by word size alone.

b) The two types of bus are: (1) dedicated bus – a bus that is permanently assigned to a specific function or component, and (2) multiplexed bus – a bus that shares lines for multiple purposes (e.g., address and data on the same lines at different times).

c) The bus is located between the CPU and memory as a set of parallel conductors (address bus, data bus,

© Hak Cipta Universiti Teknologi MARA

control bus) that connects the CPU's memory interface to the memory unit. In a simple system, the CPU and memory are connected via the system bus, with the CPU driving address and control signals and the memory responding with data.

d) System interconnection refers to the general mechanism (e.g., system bus) that provides communication among all major computer components (CPU, memory, I/O). CPU interconnection specifically refers to the internal pathways within the CPU (e.g., between registers, ALU, control unit) or the dedicated interface between the CPU and other components. The key difference is scope: system interconnection is system-wide, while CPU interconnection is local to the CPU or its immediate interfaces.

e) Drawback of synchronous bus timing: all devices must operate at the same clock rate, so the bus speed is limited by the slowest device, reducing performance. Drawback of asynchronous bus timing: it requires handshaking overhead (request/acknowledge signals), which adds complexity and can increase latency compared to synchronous timing.

Marking points:

• 1 mark: (a) States data bus is 64 bits (word size) but overall bus size also includes address and control lines; does not claim total bus size is 64 bits.

• 2 marks: (b) Lists two correct types of bus (e.g., dedicated and multiplexed) – 1 mark each.

• 2 marks: (c) Diagram or clear description showing bus between CPU and memory with address, data, and control lines – 1 mark for correct placement, 1 mark for labeling lines.

• 2 marks: (d) Differentiates system interconnection (system-wide communication among components) from CPU interconnection (internal to CPU or CPU-local) – 1 mark for each correct definition.

• 3 marks: (e) Discusses one drawback for each of two bus timing types (synchronous and asynchronous) – 1.5 marks per drawback, up to 3 marks.

Model confidence: medium.

Model note: The question part (a) is ambiguous: 'size of a bus' could mean data bus width, address bus width, or total bus width. The answer assumes word size equals data bus width. Part (c) asks for an illustration; since no figure is provided, a textual description is given. Part (e) assumes the two types are synchronous and asynchronous bus timing, which is standard in computer organization.

QUESTION 10 (10 marks)

Source: CSC429-2014-12, page [3]

Unofficial model answer:

Registers are small, high-speed storage locations inside the CPU used for temporary data and control information. Five common types are: (1) Program Counter (PC): holds the address of the next instruction to be fetched; it is automatically incremented after each fetch and can be loaded by branch/jump instructions. (2) Instruction Register (IR): holds the instruction currently being decoded and executed; its contents are split into opcode and operand fields by the control unit. (3) Memory Address Register (MAR): holds the address of the memory location to be accessed by the next read or write operation; it connects the CPU to the address bus. (4) Memory Data Register (MDR) / Memory Buffer Register (MBR): holds the data read from or to be written to memory; it connects the CPU to the data bus and acts as a buffer between the CPU and main memory. (5) Accumulator (AC): a general-purpose data register that holds one operand and the result of arithmetic/logic operations performed by the ALU. Other acceptable examples include general-purpose registers (GPRs), status/flag register (PSW), stack pointer (SP), and index registers.

Marking points:

• 1 mark: Program Counter (PC) — holds address of next instruction; incremented after fetch; loaded by branch instructions.

• 1 mark: Instruction Register (IR) — holds current instruction during decode/execute; opcode and operand fields.

• 1 mark: Memory Address Register (MAR) — holds address of memory location for read/write; drives address bus.

• 1 mark: Memory Data Register (MDR/MBR) — holds data read from or written to memory; buffers data bus.

© Hak Cipta Universiti Teknologi MARA

• 1 mark: Accumulator (AC) — holds operand and result of ALU arithmetic/logic operations.

• 1 mark: Any other valid register (GPR, status/flag/PSW, stack pointer, index register) correctly described, up to the required five.

Model confidence: high.

Model note: No figure or table was supplied; answer is based on standard computer-organization register definitions. The question asks for FIVE registers, so the marking allocates 2 marks per register (description + function) for 10 marks total; the list above gives one mark per register and one mark for a correct function, which is a common alternative allocation. Other valid registers may be accepted if correctly described.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION 11 (20 marks)

Source: CSC429-2025-07, page [5]

Unofficial model answer:

The equation in Figure 1 is read as z = (A² − B*C + jl_)/(D + 5). The OCR text is garbled ('jl_' and 'D+5' appear on a separate line), so the expression is interpreted as a numerator (A² − B*C + jl_) divided by the denominator (D + 5). Because the instruction set in Table 1 contains only ADD, SUB, MUL and DIV (no square or jump instructions), A² must be computed as A*A, and the term jl_ is treated as a single named operand/variable (call it J). Registers A, B, C, D are fixed; x, y, z are temporary, and the final result must end in z. The computation order is: t1 = A*A; t2 = B*C; t3 = t1 − t2; t4 = t3 + J; t5 = D + 5; z = t4 / t5.

(a) THREE-ADDRESS form. Each instruction names two sources and one destination, so no explicit temporaries in memory are needed beyond the named registers:

```
MUL (A, A, x)     ; x = A*A
MUL (B, C, y)     ; y = B*C
SUB (x, y, x)     ; x = A*A - B*C
ADD (x, J, x)     ; x = A*A - B*C + J
ADD (D, 5, y)     ; y = D + 5
DIV (x, y, z)     ; z = (A*A - B*C + J)/(D + 5)
```

This is the most compact form: six instructions, no PUSH/POP traffic.

(b) ONE-ADDRESS form. There is a single implicit accumulator; LOAD/STOR move between accumulator and memory, and ADD p/SUB p/MUL p/DIV p operate accumulator ← accumulator op p. Temporaries x and y hold intermediate values:

```
LOAD A        ; ACC = A
MUL A         ; ACC = A*A
STOR x        ; x = A*A
LOAD B        ; ACC = B
MUL C         ; ACC = B*C
STOR y        ; y = B*C
LOAD x        ; ACC = A*A
SUB y         ; ACC = A*A - B*C
ADD J         ; ACC = A*A - B*C + J
STOR x        ; x = numerator
LOAD D        ; ACC = D
ADD #5        ; ACC = D + 5   (immediate 5; if only memory operands allowed, STOR a constant 5 first)
STOR y        ; y = D + 5
LOAD x        ; ACC = numerator
DIV y         ; ACC = numerator/(D + 5)
STOR z        ; z = final answer
```

(c) ZERO-ADDRESS form. A stack machine: PUSH p pushes operand p, POP p pops top of stack into p, and ADD/SUB/MUL/DIV pop two operands and push the result (second-popped op top-popped). Postfix evaluation of (A*A − B*C + J)/(D + 5):

```
PUSH A
PUSH A
MUL           ; stack: A*A
PUSH B
PUSH C
```

© Hak Cipta Universiti Teknologi MARA

```
MUL         ; stack: A*A, B*C
SUB         ; stack: A*A - B*C
PUSH J
ADD         ; stack: A*A - B*C + J
PUSH D
PUSH 5
ADD         ; stack: numerator, D+5
DIV         ; stack: numerator/(D+5)
POP z       ; z = final answer
```

In all three versions the final result is stored in z, as required, and only the permitted instructions from Table 1 are used.

Marking points:

*• 2 marks: correct interpretation of Figure 1 as z = (A*A - B*C + J)/(D + 5), including handling A^2 as A*A and treating the garbled 'jl_' term as a single operand J (award partial if the denominator grouping is missed).*

• 5 marks: three-address program — correct use of MUL/SUB/ADD/DIV with (p,q,r) operand order, correct evaluation order, temporaries x and y used legally, final DIV writing to z (about 1 mark per correct instruction grouping).

• 6 marks: one-address program — correct LOAD/STOR/ADD p/SUB p/MUL p/DIV p sequence, accumulator used correctly, temporaries x and y for numerator and denominator, final STOR z (deduct for missing STOR or wrong operand order).

• 6 marks: zero-address program — correct postfix ordering with PUSH/POP/ADD/SUB/MUL/DIV, stack discipline correct (two pops, one push per arithmetic op), final POP z.

• 1 mark: overall correctness — all three programs compute the same expression and leave the answer in z, using only instructions listed in Table 1.

Model confidence: medium.

*Model note: Figure 1 is supplied only as a cropped image with garbled OCR text ('z=A2-B*C+jl_' and 'D+5'); the exact numerator term (appearing as 'jl_') and the division grouping are inferred. If the intended expression differs (e.g. the denominator applies only to part of the numerator, or 'jl_' is two operands j and l), the instruction sequences must be adjusted accordingly. The one-address immediate 'ADD #5' assumes an immediate form; if only memory operands are permitted, a constant 5 must be stored to a temporary first. This is an unofficial model answer, not an official UTM marking scheme.*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA